

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 3

Implementation and Analysis of a Convolutional Neural Network for CIFAR-10 Image Classification

1. Objective

The objective of this experiment is to understand the fundamental concepts involved in designing and implementing Convolutional Neural Networks (CNNs) for image classification using the TensorFlow/Keras deep learning framework. The experiment focuses on convolutional layers, activation functions, pooling operations, flattening, fully connected layers and the influence of important hyperparameters on CNN performance.

A CNN is constructed and trained using the CIFAR-10 image dataset containing ten object categories. The experiment also investigates learned feature representations through intermediate feature-map visualization and evaluates the trained model using accuracy, precision, recall, F1-score, confusion matrix and classification report.

2. Tasks to be Performed

1. Understand the basic building blocks of CNNs.
2. Load and inspect the CIFAR-10 dataset.
3. Perform basic exploratory analysis.
4. Preprocess image data and class labels.
5. Design a CNN using TensorFlow/Keras.
6. Investigate kernel size, stride, padding and number of filters.
7. Study optimizer, batch size and training epochs.
8. Monitor training and validation performance.
9. Visualize intermediate convolutional feature maps.
10. Compare pooling strategies.
11. Analyze trainable parameters.
12. Evaluate accuracy, precision, recall and F1-score.
13. Generate and interpret the confusion matrix.
14. Analyze sample predictions and classification errors.

3. Background Theory

A Convolutional Neural Network is a deep learning architecture designed primarily for structured grid-like data such as images. CNNs automatically learn useful visual features directly from image pixels. A typical CNN contains convolutional layers, activation functions, pooling layers, flattening operations and fully connected layers.

3.1 Convolutional Layer

A convolutional layer applies learnable filters to an input image or feature map. Each filter slides over the spatial dimensions and computes local weighted combinations:

$$Z(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n) + b.$$

Here, X denotes the input feature map, K the convolution kernel, b the bias and Z the output feature map. Different filters can learn edges, textures, corners and object parts.

3.2 Activation Function

The convolution operation is followed by a nonlinear activation function. The Rectified Linear Unit (ReLU) is widely used in CNNs:

$$f(x) = \max(0, x).$$

It retains positive values and replaces negative values with zero.

3.3 Pooling

Pooling reduces the spatial dimensions of feature maps while retaining important information. Max Pooling selects the largest activation within a local region:

$$P(i, j) = \max_{(m, n) \in W} X(i + m, j + n).$$

Pooling reduces computational requirements and provides a degree of translational robustness.

3.4 Flattening and Fully Connected Layers

After convolution and pooling, the feature maps are converted into a one-dimensional vector using flattening. The resulting representation is passed to dense layers that combine the extracted features for classification.

3.5 Softmax Output

For ten-class classification, the final layer can use Softmax:

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}.$$

The class with the highest probability becomes the final prediction.

4. CNN Architecture

The CNN developed for this experiment uses multiple convolutional stages followed by pooling and fully connected layers. The number of filters progressively increases as

$$32 \rightarrow 64 \rightarrow 128,$$

while the spatial dimensions decrease approximately as

$$32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8 \rightarrow 4 \times 4.$$

Stage	Filters	Operation	Representation
1	32	Convolution + Pooling	Low-level features
2	64	Convolution + Pooling	Intermediate features
3	128	Convolution + Pooling	Higher-level features
4	–	Flatten + Dense	Classification

5. Dataset Description

The CIFAR-10 dataset is used for multi-class image classification. It contains colour images of ten object categories. Each image has resolution 32×32 pixels and three RGB channels.

Property	Description
Dataset	CIFAR-10
Image Size	32×32
Channels	3 RGB channels
Number of Classes	10
Training Images	50,000
Test Images	10,000
Task	Multi-class Classification

The ten classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck.

6. Data Preprocessing

The CIFAR-10 pixel values are normalized before training:

$$\mathcal{X}_{\text{normalized}} = \frac{\mathcal{X}}{255}.$$

This converts the original range $[0, 255]$ to $[0, 1]$. The dataset is divided into training and validation portions during model development, while the official test set is retained for final evaluation.

7. Exploratory Data Analysis

Exploratory analysis was performed to understand the image data and class distribution. The analysis included sample image inspection, number of classes, class distribution, image dimensions and representative CIFAR-10 images. The dataset contains visually diverse categories, with several classes having similar visual characteristics.

8. CNN Hyperparameters

Several hyperparameters were investigated during the experiment.

Hyperparameter	Values / Configuration
Kernel Size	$3 \times 3, 5 \times 5$
Filters	32, 64, 128
Stride	1, 2
Padding	Same, Valid
Pooling	Max Pooling and alternative strategy
Optimizer	Adam, SGD
Batch Size	32, 64
Epochs	As selected during training

These parameters influence computational cost, representation capacity, convergence behaviour and generalization.

9. Model Architecture

The CNN contains three principal convolutional feature-extraction stages. The first stage contains 32 filters, the second 64 and the third 128.

Input $\rightarrow$ Conv(32) $\rightarrow$ Pooling $\rightarrow$ Conv(64) $\rightarrow$ Pooling $\rightarrow$ Conv(128) $\rightarrow$ ~~ReLU~~ Flatten $\rightarrow$ Dense $\rightarrow$ Output.

The output layer contains ten neurons corresponding to the CIFAR-10 classes.

10. Training Configuration

The CNN was trained using TensorFlow/Keras. Training and validation accuracy and loss were recorded after each epoch. These curves were used to determine whether the model learned effectively and whether a difference between training and validation performance was present.

11. Training and Validation Accuracy

The training accuracy increases as the CNN learns useful representations from the training images. The documented best training accuracy was approximately **89.65%**, while the best validation accuracy was approximately **75.57%**. The difference indicates a generalization gap between training and validation performance.

12. Training and Validation Loss

The loss curves provide another perspective on the training behaviour. A decreasing training loss indicates improvement in the fit to training samples. The relationship between training and validation loss provides information about the generalization behaviour of the network.



Figure 1: Training and validation accuracy during CNN training.

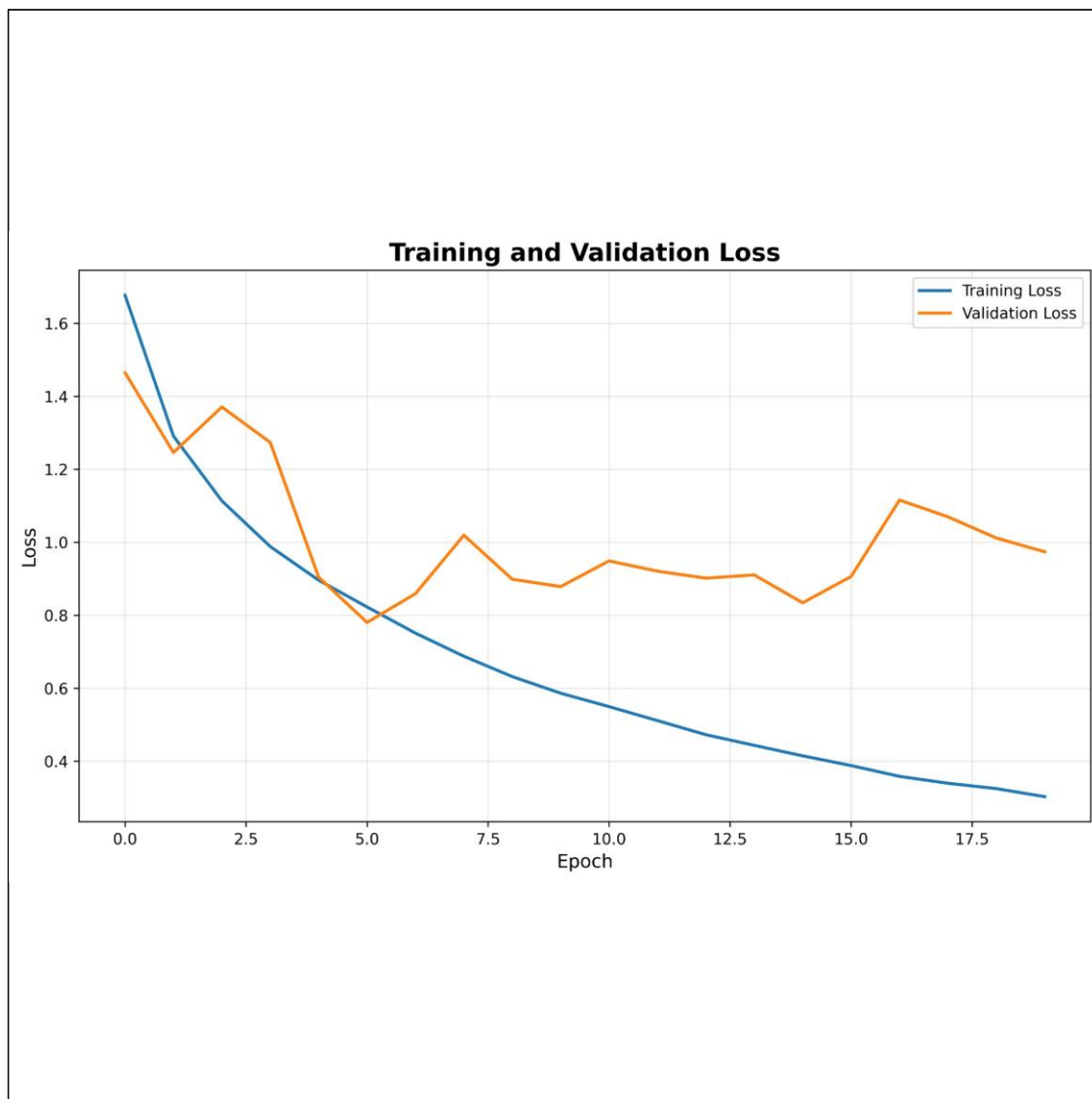


Figure 2: Training and validation loss during CNN training.

13. Feature Map Visualization

Feature maps provide intermediate representations of an image as it passes through convolutional layers. Their visualization gives insight into the information learned at different depths. Early layers generally respond to local structures, whereas deeper layers combine these patterns into more complex representations.

14. Feature Maps from Convolutional Layer 1

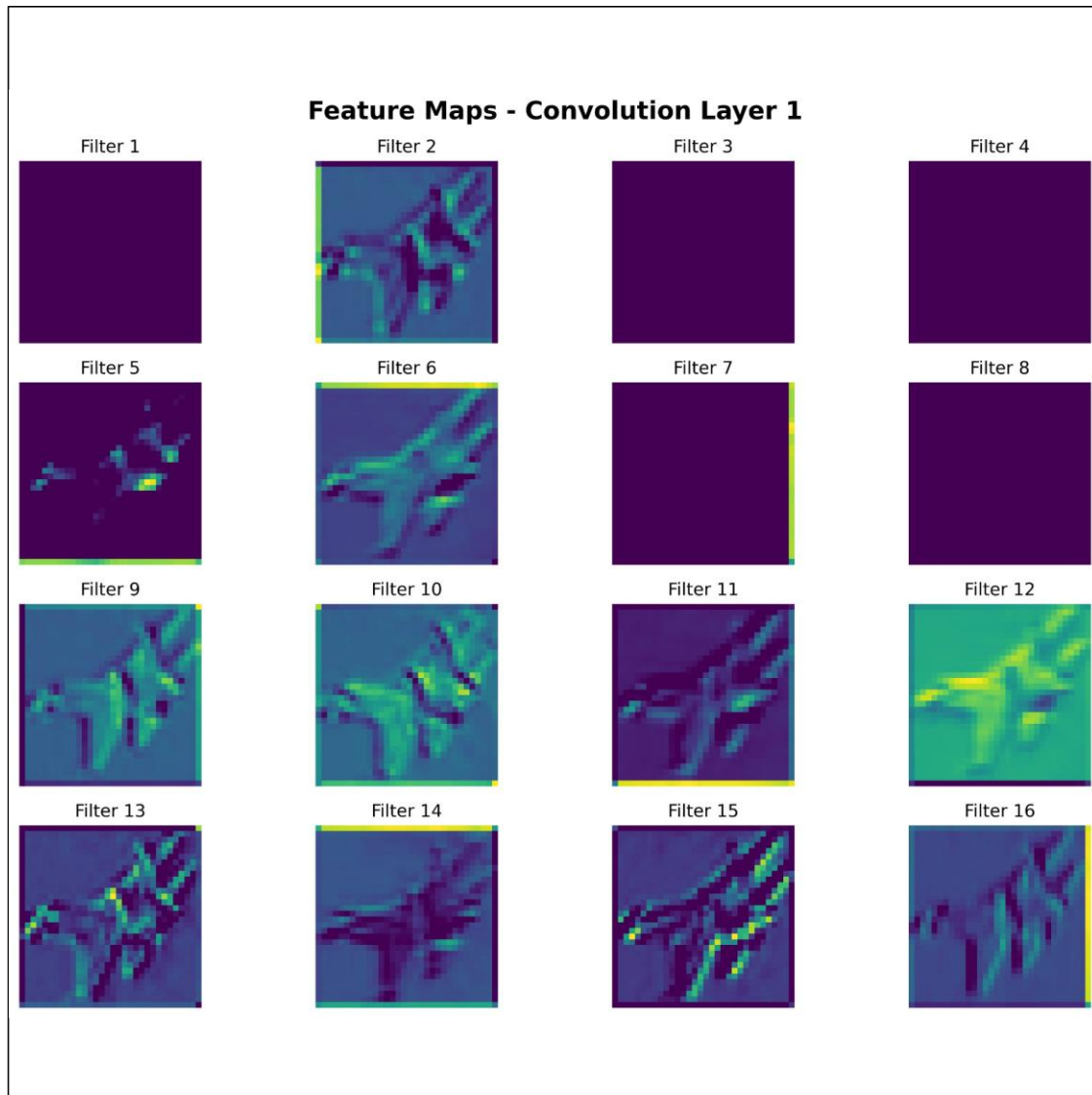


Figure 3: Feature maps generated by the first convolutional layer.

The first convolutional layer produces relatively low-level visual representations. The activations can highlight local edges, intensity transitions, simple textures and other basic image structures.

15. Feature Maps from Convolutional Layer 2

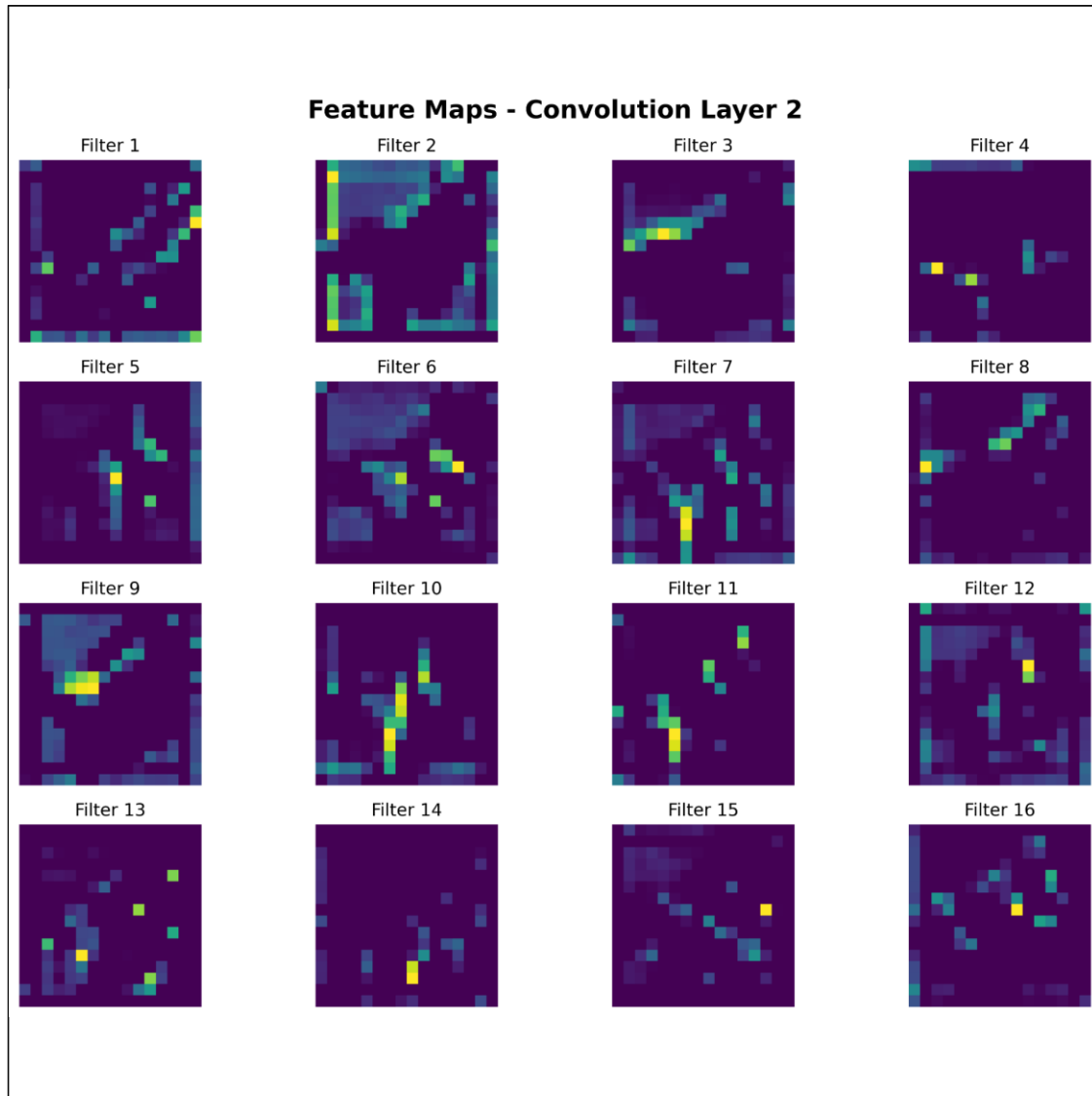


Figure 4: Feature maps generated by the second convolutional layer.

The second convolutional layer receives representations generated by the previous layer and learns more complex combinations of local patterns. Therefore, these feature maps represent intermediate-level structures rather than only individual edges or simple textures.

16. Feature Maps from Convolutional Layer 3

The third convolutional layer represents a deeper stage of the feature hierarchy. Its activations correspond to increasingly abstract combinations of visual patterns learned from the preceding layers.

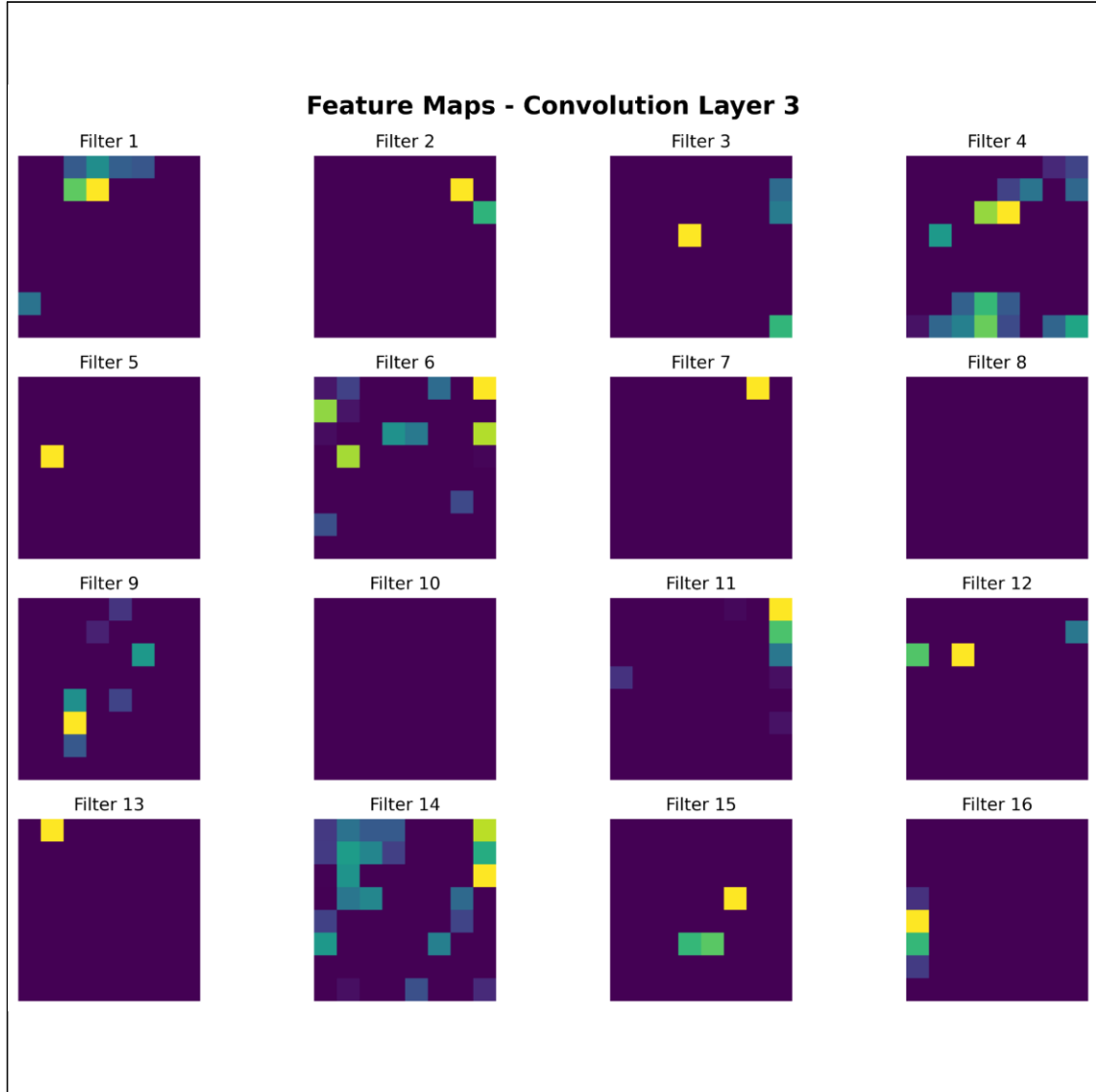


Figure 5: Feature maps generated by the third convolutional layer.

17. Pooling Strategy Comparison

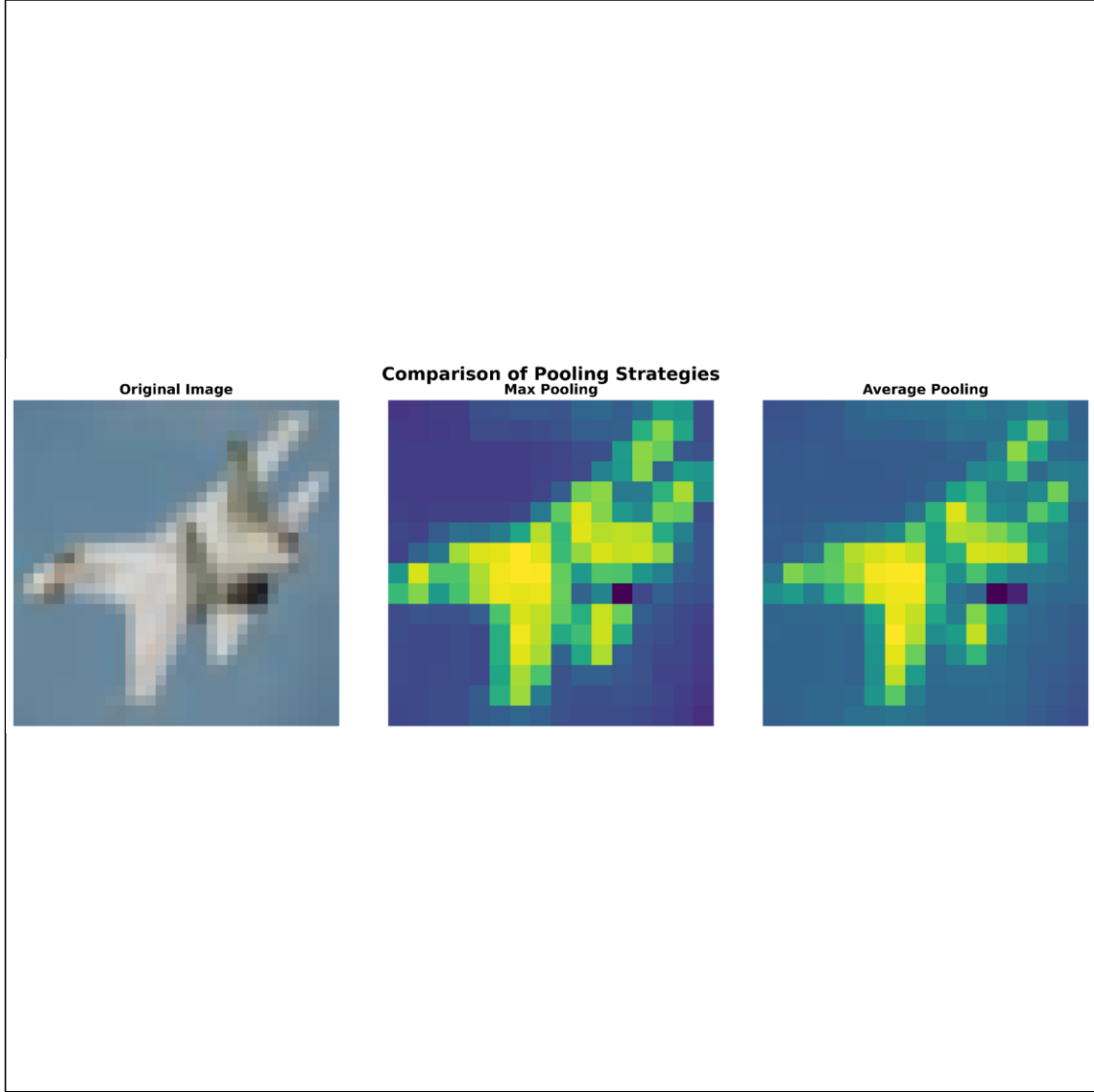


Figure 6: Comparison of pooling strategies used during the experiment.

Pooling reduces feature-map dimensions while retaining important information. Max Pooling selects the strongest activation within a local region and therefore emphasizes the strongest detected feature. The comparison demonstrates how pooling strategy changes the spatial representation passed to subsequent layers.

18. Trainable Parameter Analysis

The number of trainable parameters in a convolutional layer depends on the kernel dimensions, input channels and output filters:

$$N = (K_h \times K_w \times C_{in} + 1) \times C_{out}.$$

Here, K_h and K_w denote kernel height and width, C_{in} the number of input channels and C_{out} the number of output filters. The additional 1 accounts for the bias parameter. Increasing the number of filters from 32 to 64 and then 128 increases representation capacity and model complexity.

19. Confusion Matrix

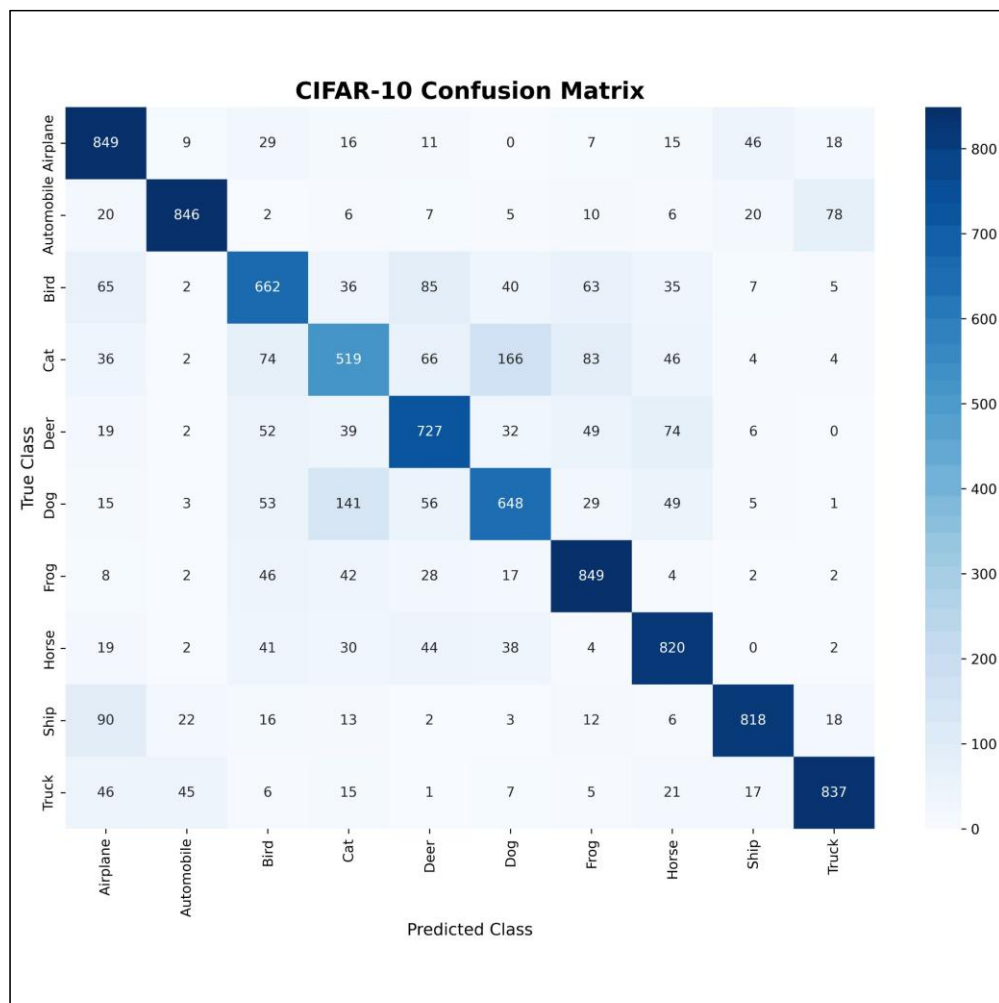


Figure 7: Confusion matrix for the CIFAR-10 test set.

The diagonal entries represent correctly classified images, whereas off-diagonal entries represent misclassifications. Noticeable errors can occur between visually similar categories, indicating that visual similarity remains an important source of classification error.

20. Sample Predictions and Error Analysis

Prediction analysis provides a qualitative complement to numerical evaluation. Correct predictions demonstrate that the learned representation captures useful object-level patterns. Incorrect predictions may occur when categories share similar shapes, textures, colours or low-resolution visual structures.

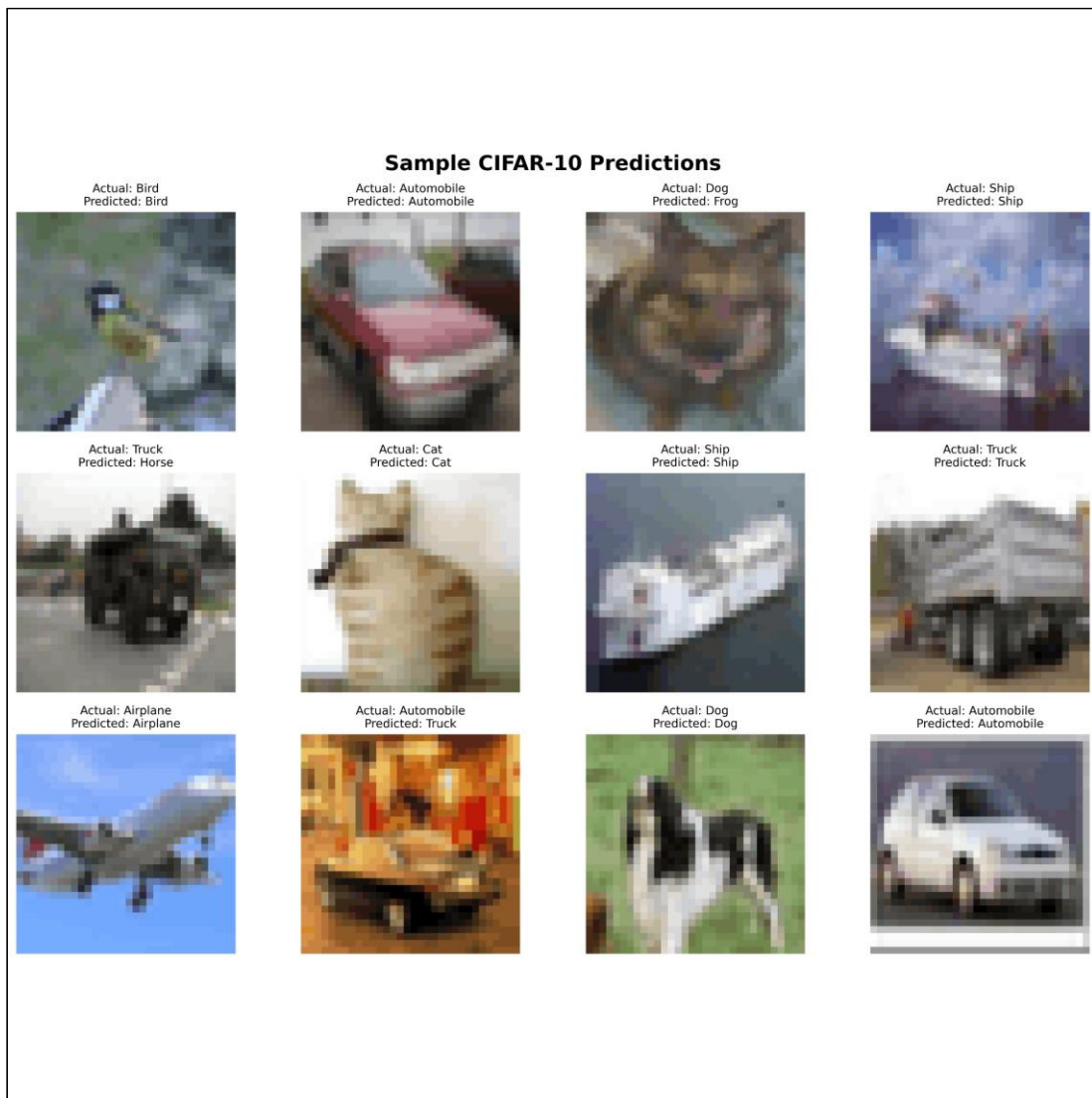


Figure 8: Representative CIFAR-10 predictions generated by the trained CNN.

21. Consolidated Hyperparameter Results

Hyperparameter	Compared Values	Observed Result
Kernel Size	3×3 , 5×5	3×3 performed better
Filters	(16, 32), (32, 64)	More filters performed better
Optimizer	Adam, SGD	Adam performed better
Batch Size	32, 64	Batch size 32 performed better
Stride	1, 2	Larger stride reduced spatial resolution
Padding	Same, Valid	Valid reduced spatial dimensions

The hyperparameter experiments were conducted as controlled short-duration comparisons using a smaller subset of the dataset. Their purpose was to investigate how architectural and optimization choices influence CNN behaviour rather than replace the final model evaluation.

22. Discussion

The experiment demonstrates the complete workflow of applying a CNN to a multi-class image classification problem. The network uses convolutional layers to learn local visual structures, pooling to reduce spatial dimensions and dense layers to map learned representations to the ten CIFAR-10 classes.

A major observation is the increase in representational capacity with network depth. The convolutional layers use 32, 64 and 128 filters while the spatial resolution decreases progressively. This arrangement allows the network to exchange spatial detail for increasingly rich channel-wise representations.

The feature-map analysis provides an interpretable view of this process. Early activations preserve more local information, whereas deeper activations represent more abstract combinations of patterns.

The training curves indicate that the model fits the training data better than the validation data. The difference between the best training accuracy of 89.65% and best validation accuracy of 75.57% indicates a noticeable generalization gap.

The controlled hyperparameter experiments further demonstrate that CNN performance is sensitive to architectural and optimization choices.

23. Limitations and Scope for Improvement

Although the CNN achieved useful classification performance, several improvements could be investigated:

1. Data augmentation could improve generalization.
2. Early stopping could reduce unnecessary training.
3. Learning-rate scheduling could stabilize optimization.

4. A deeper architecture or transfer-learning model could be evaluated.
5. More systematic hyperparameter search could be performed.
6. Class-specific errors could be reduced through stronger representations.

24. Conclusion

This experiment provided practical experience in designing and analyzing a Convolutional Neural Network for image classification using TensorFlow/Keras. A CNN was constructed and trained on CIFAR-10 using convolutional layers, batch normalization, Max Pooling, flattening, dense layers and dropout.

The model achieved approximately 75.75% test accuracy in the documented evaluation. The best documented training accuracy was approximately 89.65%, while the best validation accuracy was approximately 75.57%.

The feature-map investigation demonstrated the hierarchical nature of CNN representations, while the pooling experiment illustrated the effect of spatial aggregation. Parameter analysis showed how filters and input channels influence model complexity.

Overall, the experiment established the relationship between CNN architecture, learned visual features, optimization behaviour and classification performance, providing a practical foundation for further computer vision and deep learning work.

25. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
 2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
 3. F. Chollet, *Deep Learning with Python*, Manning Publications.
 4. A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," University of Toronto, 2009.
 5. TensorFlow/Keras Documentation: <https://www.tensorflow.org/>
 6. CIFAR-10 Dataset: <https://www.cs.toronto.edu/~kriz/cifar.html>
 7. Scikit-learn Documentation: <https://scikit-learn.org/>
-